

SchoolSync

Production Migration Runbook

*From localhost to a managed, multi-tenant,
production-shaped stack on Render + Supabase + Vercel.*

ARCHITECTURE · SECURITY · DEPLOY · OPERATE

TARGET

theExceptionalSam/oo (NestJS modular monolith)

~75 USD / month managed stack

Multi-tenant · RLS · RS256 · Graphile Worker · Supabase Realtime

AUDIENCE

Solo founder / backend lead / first hire

Familiarity with TypeScript, Docker, and Postgres assumed.

Table of Contents

1. Executive Summary	4
2. Target Architecture	4
2.1 What was removed and why	5
3. Migration Sequence (12 Weeks)	5
4. Phase 1 — Cut Dead Weight (Weeks 1–2)	6
4.1 Drop Elasticsearch	6
4.2 Drop MinIO, move to R2	7
5. Phase 2 — Move Data to Supabase (Weeks 3–4)	8
5.1 Create the Supabase project	8
5.2 Run migrations against Supabase	8
6. Phase 3 — RLS Policies (Weeks 5–6)	9
6.1 How the RLS mechanism works	9
6.2 The middleware that sets the context	9
6.3 SUPER_ADMIN bypass	9
7. Phase 4 — Graphile Worker (Weeks 7–8)	11
7.1 Why Postgres-as-queue is safe	11
7.2 Migration pattern	11
8. Phase 5 — Supabase Realtime (Weeks 9–10)	12
8.1 Frontend subscription pattern	12
9. JWT Hardening — RS256 Only	13
9.1 Generating a keypair	13
9.2 Bootstrap admin (no more seed)	13
10. Deploy to Render + Vercel + Cloudflare	14
10.1 Deploy sequence	14
10.2 Render configuration	14
10.3 Vercel configuration	15
10.4 Cloudflare configuration	15
11. Observability — The First 48 Hours	16
11.1 Uptime monitoring	16
11.2 Error tracking (Sentry)	16

- 11.3 Database backup verification 16
- 11.4 Reboot test 16
- 12. Cost Summary 17**
- 13. Operational Runbook — First Incidents 18**
 - 13.1 Cross-tenant data leak suspected 18
 - 13.2 Database unavailable 18
 - 13.3 Payment provider outage 18
- 14. The 14-Day Self-Test 19**

1. Executive Summary

This runbook walks you through the full migration of the SchoolSync codebase (currently a NestJS modular monolith with Postgres, Redis, MinIO, Elasticsearch, BullMQ, and Socket.io) onto a managed, production-shaped stack. The target stack consolidates five stateful services into one (Postgres on Supabase) and four stateless compute layers (Vercel for the SPA, Render for the API, Render for the worker, Cloudflare for edge DNS/CDN/R2 storage).

The migration has three goals, in priority order. First, eliminate the silent scaling cliff at ~50 schools by adding Postgres Row-Level Security (RLS) so a missed `WHERE school_id = ?` clause in any future query can never leak data across tenants. Second, remove every stateful service that is not pulling its weight at your scale — Elasticsearch (replaced by Postgres tsvector + GIN indexes), MinIO (replaced by Cloudflare R2), Redis + BullMQ (replaced by Graphile Worker on the same Postgres), and Socket.io (replaced by Supabase Realtime). Third, harden authentication by killing the HS256 dev fallback and forcing RS256 only, with key IDs for rotation.

The resulting architecture scales to roughly one thousand schools without a rewrite, costs about seventy-five US dollars per month, and removes the operational burden of running Redis, Elasticsearch, and MinIO yourself. The trade-off is mild vendor lock-in on Supabase for the stateful layer, mitigated by the fact that every component speaks an open standard (Postgres wire protocol, S3 API, Docker images).

PREREQUISITE

This runbook assumes the migration pack files in `schoolsync-prod-pack/` have been copied into your local clone of the repo. Each file is referenced by its target path in the repo.

2. Target Architecture

The target architecture follows one principle: **Postgres-first, everything else stateless**. Postgres becomes the database, the queue, the search engine, and the realtime source of truth. Every other component in the stack is stateless compute that can be replaced, scaled, or moved without data migration.

The diagram below shows the five components and the data flow between them. Notice what is missing: Redis, MinIO, Elasticsearch, and the Socket.io server are all gone. They are not needed at your scale, and removing them eliminates four categories of operational failure (memory pressure on Redis, disk pressure on MinIO, JVM heap on Elasticsearch, websocket scaling on Socket.io).

Layer	Service	Role	Cost / mo
Edge	Cloudflare	DNS, CDN, WAF, DDoS protection	Free
Edge storage	Cloudflare R2	S3-compatible object storage (zero egress)	\$0–2
Frontend	Vercel	React/Vite static SPA, global CDN, preview deploys	\$0–20

Layer	Service	Role	Cost / mo
API	Render (Web Service)	NestJS modular monolith, stateless	\$25
Worker	Render (Background Worker)	Graphile Worker, Postgres-backed queue	\$25
Database	Supabase (Pro)	Postgres 15 + RLS + PITR + Realtime + Storage	\$25
Rate limit / cache	Upstash	Serverless Redis (optional — ScopeService cache only)	\$0–5
		Total	~\$75–100

2.1 What was removed and why

Each removed service was a deliberate trade-off. The reasoning matters because at some future scale you may need to re-introduce one of them, and you should know the trigger point that justifies the re-introduction.

Service	Removed because	Re-add when
Elasticsearch	Search workload is "find student by name in 800 rows". Postgres tsvector + GIN handles this in 2ms.	You exceed 1M rows per tenant or need faceted search across 10+ dimensions.
Redis + BullMQ	Peak queue throughput is ~10 jobs/sec. Graphile Worker on Postgres handles ~1k/sec.	You exceed 500 jobs/sec sustained, or need pub/sub fan-out to thousands of subscribers.
MinIO	Object storage should be a managed S3-compatible API, not a service you operate. R2 has zero egress fees.	Never. R2 is strictly better for this use case.
Socket.io server	Realtime workload is ~1 event per user per minute. Supabase Realtime streams Postgres CDC natively.	You need custom realtime events that do not correspond to DB changes (e.g., collaborative editing).
HS256 JWT fallback	A misconfigured env var in prod silently degrades auth to a symmetric key. This is a security footgun.	Never. RS256 only is a hard rule.

3. Migration Sequence (12 Weeks)

The migration is sequenced so that each phase is independently shippable. You can stop after any phase and have a working system — the next phase simply improves it. This is critical for a solo founder: you should never be in a state where you have to ship all twelve weeks of work before the product works again.

STRATEGY

The phases are ordered by **risk and reversibility**. The lowest-risk, most-reversible changes come first. The riskiest change (RLS policies) sits in the middle, after you have a working backup and observability in place to catch any cross-tenant leak.

Phase	Weeks	Goal	Risk	Reversible
1. Cut dead weight	1–2	Drop ES + MinIO. Migrate search to tsvector, storage to R2.	Low	Yes (one env var)
2. Move data to Supabase	3–4	Create Supabase project, point TypeORM at pooler URL, verify.	Medium	Yes (pg_dump restore)
3. RLS policies	5–6	Enable RLS on every tenant-owned table. Add SET LOCAL middleware.	High	Partial (policy drop is easy; column drop is not)
4. Replace BullMQ with Graphile	7–8	Migrate job definitions 1:1. Dual-write for 1 week. Cut over.	Medium	Yes (revert to BullMQ)
5. Replace Socket.io with Supabase Realtime	9–10	Swap client SDK. Update backend emit calls.	Medium	Yes (keep Socket.io server running)
6. Deploy + observe	11–12	Ship to Render + Vercel. Run for 2 weeks with yourself as only user.	Low	N/A (final phase)

4. Phase 1 — Cut Dead Weight (Weeks 1–2)

The first phase removes Elasticsearch and MinIO from the stack. These are the lowest-risk changes because they have well-defined replacements (Postgres tsvector and Cloudflare R2 respectively) and the migration is reversible by changing one environment variable. Doing this first shrinks the surface area for every subsequent phase.

4.1 Drop Elasticsearch

The current `SearchService` in `src/shared/search/search.service.ts` is an in-memory substring matcher — it has never actually used Elasticsearch in dev. The replacement file in the migration pack is a real Postgres-backed implementation that uses `tsvector` columns and GIN indexes created by migration `17800000000001-AddPostgresSearch.ts`.

The migration adds `search_vector` generated columns to five tables (students, users, classes, subjects, announcements) plus trigram indexes on string columns for fuzzy `ILIKE` matching. This combination handles every realistic search query at your scale:

- Find a student by roll number — trigram index on `students.roll_number`, 2ms.
- Find a user by email — trigram index on `users.email`, 2ms.

- Find a class by name — `tsvector` on `classes.name + grade_level + section`, 3ms.
- Full-text search announcements — `tsvector` on `title + content`, 5ms.

The `indexDocument` and `remove` methods are kept as no-ops for API parity with the old service — the `search_vector` column is GENERATED ALWAYS AS, so it updates automatically when the underlying columns change. No application code needs to manage the index.

4.2 Drop MinIO, move to R2

The current `StorageService` is also an in-memory mock — bytes are stored in a `Map` and lost on every deploy. The replacement file uses the AWS SDK v3 S3 client, which is wire-compatible with Cloudflare R2, AWS S3, Supabase Storage, Backblaze B2, and any other S3-compatible endpoint. The choice of provider is purely an environment variable change.

WHY R2

Cloudflare R2 is the recommended production target because it has **zero egress fees**. AWS S3 charges \$0.09 per GB of egress; for a school SaaS where teachers download report card PDFs and student photos regularly, this becomes the largest line item on your AWS bill within months.

The new `StorageService` adds three capabilities the old mock lacked: server-side presigned PUT URLs (so clients upload directly to R2, bypassing your API server and saving bandwidth), presigned GET URLs for time-limited downloads, and a real SHA-256 checksum for deduplication. The interface is otherwise identical to the old service, so call sites in feature modules do not need to change.

5. Phase 2 — Move Data to Supabase (Weeks 3–4)

Phase 2 moves Postgres off your localhost / VPS and onto Supabase Pro. This is the foundational change that every subsequent phase depends on: Supabase gives you Row-Level Security (RLS), point-in-time recovery (PITR), Realtime (CDC over websockets), and Storage (S3-compatible) — all on the same Postgres instance.

5.1 Create the Supabase project

Sign up at supabase.com, create a new project, and pick the Frankfurt region (closest to Nigeria with good peering). Set a strong database password and store it in a password manager immediately — Supabase does not show it again. On the free tier the database pauses after one week of inactivity, which is unacceptable for a school product, so upgrade to Pro (\$25/month) before onboarding any real school.

After project creation, navigate to Project Settings → Database. You will see two connection strings: a direct URL (port 5432) and a pooler URL (port 6543). Use the pooler URL in production. The direct URL has a hard cap of 60 connections; the pooler multiplexes thousands of application connections through a small PgBouncer pool, so it is the only safe choice for a multi-instance deployment.

COMMON MISTAKE

Set `DB_POOL_MAX=5` in your environment. A small pool combined with PgBouncer multiplexing is the correct configuration. A large pool (`DB_POOL_MAX=20`) will exhaust the Supabase pooler and cause connection errors across all instances.

5.2 Run migrations against Supabase

Point your local NestJS `.env` at the Supabase pooler URL and run the existing migrations plus the three new ones from the migration pack:

```
DATABASE_URL=postgresql://postgres.xxx:password@aws-0-frankfurt.pooler.supabase.com:6543/postgres \
DB_SSL=true DB_POOL_MAX=5 npm run migration:run
```

The three new migrations are applied in order: first the RLS policies (`1780000000000-AddRlsPolicies.ts`), then the Postgres search indexes (`1780000000001-AddPostgresSearch.ts`), then the Graphile Worker schema (`1780000000002-AddGraphileWorkerSchema.ts`). All three are idempotent — safe to re-run. Verify in the Supabase dashboard that the `tenant_isolation` policy exists on every tenant-owned table.

After migrations succeed, run the bootstrap admin script (covered in Section 8) instead of the old `npm run seed`. The seed script created the demo school with credentials `admin@demo-school.edu / Demo!Pass123` — these are on every credential-stuffing list and must never exist in production.

6. Phase 3 — RLS Policies (Weeks 5–6)

Phase 3 is the highest-risk and highest-impact change in the migration. It moves tenant isolation from the application layer (where a missed `WHERE school_id = ?` predicate in any future query leaks data) to the database layer (where the policy enforces isolation on every query, including ones not yet written).

CRITICAL

Test RLS aggressively. Run your existing `tenant-test.mjs` script after applying the policies. If any test returns data from another tenant, the policy has a bug — do NOT ship. Expand the test to 300+ randomised cross-tenant access attempts before considering the phase complete.

6.1 How the RLS mechanism works

Each request opens a transaction and sets a Postgres session variable:

```
SET LOCAL app.current_school_id = '<school-uuid>';

-- Now every query against a tenant-owned table is automatically scoped:
SELECT * FROM students; -- only returns students where school_id matches
SELECT * FROM attendance WHERE date = '2026-08-28'; -- already scoped
-- Even a buggy query that forgets WHERE school_id = ? cannot leak:
SELECT * FROM users; -- returns only users in the current school
```

The `SET LOCAL` only applies inside the current transaction. A connection-level `SET` would leak across requests in the pool, which is why the `RlsContextMiddleware` in the migration pack opens a per-request transaction. The overhead is roughly fifty microseconds per request — negligible compared to typical endpoint latency of five to fifty milliseconds.

6.2 The middleware that sets the context

The `RlsContextMiddleware` in `src/common/middleware/rls-context.middleware.ts` runs before every controller. It reads `req.tenant.schoolId` (set by the existing `TenantContextGuard` from the X-School-Subdomain header), opens a transaction, sets the session variable, and exposes the query runner on `req.dbQueryRunner` for services that want to use it explicitly.

The existing `SchoolContextGuard` remains as defense-in-depth. If a future developer adds a route that bypasses the middleware (e.g., a CLI script or a background worker), the guard still catches cross-tenant access at the application layer. The two layers exist independently — a bug in one does not create a vulnerability.

6.3 SUPER_ADMIN bypass

Cross-tenant admin operations (super-admin viewing any school, platform-level reports) need to bypass RLS. The middleware handles this by setting a different session variable for `SUPER_ADMIN` requests:

```
SET LOCAL app.bypass_qls = 'on';

-- Combined with the BYPASSRLS role attribute on the schoolsync_app role,
-- SUPER_ADMIN queries return rows from all tenants.
```

This is intentional and audited. Every `SUPER_ADMIN` query writes an entry to `audit_logs` with the actor, target school, and payload — so even though RLS is bypassed, every cross-tenant access is logged and reviewable. The audit log itself is RLS-protected (a school can read its own audit entries, but never another school's).

7. Phase 4 — Graphile Worker (Weeks 7–8)

Phase 4 replaces BullMQ + Redis with Graphile Worker on the same Postgres database. This is the change that removes Redis from your stack entirely. The trade-off is a small latency increase (Postgres is microseconds, Redis is microseconds — both are dominated by network round-trip at your scale) in exchange for eliminating one stateful service and gaining transactional outbox semantics for free.

7.1 Why Postgres-as-queue is safe

BullMQ on Redis can handle roughly ten thousand jobs per second. Graphile Worker on Postgres handles roughly one thousand jobs per second on a single instance. SchoolSync peaks at about ten jobs per second (notification fan-out at 8:05 AM when teachers take attendance). You have one hundred times headroom before hitting the ceiling. The latency difference between Redis and Postgres for a single job enqueue is roughly two hundred microseconds — irrelevant for any human-noticeable workflow.

The benefit that justifies the change is **transactional outbox semantics**. When you enqueue a job inside a database transaction, the job is only visible to workers after the transaction commits. This eliminates the "job queued, but DB write failed" race condition that plagues every BullMQ deployment. With BullMQ, you have to write compensating logic to handle this; with Graphile Worker, it is impossible by construction.

7.2 Migration pattern

The migration is mechanical. Each BullMQ job definition becomes a Graphile task function with the same signature. The `EventBus` service in the migration pack wraps the new queue and exposes the same `publish(event, payload)` interface, so call sites in feature modules do not need to change.

Run both queues in parallel for one week. Dual-write every event to both BullMQ and Graphile Worker, with both processors active. Compare the audit logs after a week — if Graphile has processed one thousand jobs cleanly with no drift from BullMQ, cut over by removing the BullMQ writer. The dual-write period is your safety net: if Graphile has a bug, you have not lost any jobs.

8. Phase 5 — Supabase Realtime (Weeks 9–10)

Phase 5 replaces the Socket.io server with Supabase Realtime. The current `NotificationsProcessor` emits events via Socket.io, which requires a Redis adapter when scaled to multiple instances. Supabase Realtime streams Postgres changes (Change Data Capture) over websockets directly from the database — when a row is inserted into the `notifications` table, every connected client subscribed to that school receives the event automatically.

The `RealtimeService` in `src/shared/realtime/realtime.service.ts` is a thin wrapper around the Supabase client. It exposes two methods: `broadcastToSchool` and `broadcastToUser`. These are only needed for events that do not correspond to a DB insert (e.g., "report card generation complete" — a worker-side event). For DB-backed events (announcements, messages, notifications), the frontend subscribes directly to the table changes and no backend code is needed at all.

8.1 Frontend subscription pattern

```
// _oe/src/lib/realtime.ts
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY,
);

export function subscribeToNotifications(schoolId: string, onNew: (n: any) => void) {
  return supabase
    .channel(`school:${schoolId}`)
    .on('postgres_changes',
      {
        event: 'INSERT',
        schema: 'public',
        table: 'notifications',
        filter: `school_id=eq.${schoolId}`,
      },
      (payload) => onNew(payload.new),
    )
    .subscribe();
}
```

The filter clause ensures the client only receives notifications for its own school. RLS on the `notifications` table is a second defense-in-depth layer — even if a client manipulated the filter, RLS would prevent them from receiving rows from another tenant.

9. JWT Hardening — RS256 Only

The current `JwtConfigService` falls back to HS256 if no RSA keypair is configured. This is a security footgun: a misconfigured environment variable in production silently degrades authentication to a symmetric signing algorithm, where anyone who leaks `JWT_SECRET` can forge tokens. The replacement file in the migration pack removes the fallback entirely.

The new `JwtConfigService` requires an RSA keypair in every environment except the test environment. In production, it hard-fails at boot if no keypair is found. In development, it prints clear instructions for generating one with OpenSSL. The `deploy/gen-dev-keys.sh` script in the migration pack automates this for local development.

9.1 Generating a keypair

```
# Local dev keypair
mkdir -p keys
openssl genrsa -out keys/private.pem 2048
openssl rsa -in keys/private.pem -pubout -out keys/public.pem

# Production: load keys from your secrets manager, not from disk.
# Render: paste the full PEM (multi-line) into a Secret env var.
# Set JWT_PRIVATE_KEY and JWT_PUBLIC_KEY (not _PATH) in the env.
```

The new config also derives a stable key ID (`kid`) from the SHA-256 fingerprint of the public key, included in the JWT header. This enables key rotation without invalidating in-flight tokens: deploy a new keypair with a new `kid`, and the verification side accepts both `kids` for a grace period before removing the old one.

9.2 Bootstrap admin (no more seed)

The old `npm run seed` script created a demo school with hardcoded credentials. The replacement `npm run bootstrap:admin` script in `src/database/seeds/bootstrap-admin.ts` prompts for or generates a random password, creates the school and admin user in a single transaction, prints the credentials once, and forces password change on first login. In production it refuses to run if the school already has an admin.

10. Deploy to Render + Vercel + Cloudflare

Once phases 1–5 are complete and the test suite is green, deploy to the managed stack. The deployment sequence matters: the database must exist before the API can boot, the API must be reachable before the frontend can talk to it, and the DNS records must be in place before you can verify end-to-end.

10.1 Deploy sequence

Step	Action	Time	Blocker
1	Create Supabase Pro project (Frankfurt region).	5 min	—
2	Run migrations against Supabase via local NestJS.	10 min	Step 1
3	Run <code>bootstrap:admin</code> to create first school + admin.	2 min	Step 2
4	Create Cloudflare R2 bucket + API token.	5 min	—
5	Create Upstash Redis (free tier) — optional.	3 min	—
6	Push repo to GitHub (if not already).	1 min	—
7	In Render: New → Blueprint → connect repo. Pick <code>render.yaml</code> .	10 min	Steps 1, 4, 5
8	Set all env vars marked <code>sync:false</code> in <code>render.yaml</code> (dashboard).	15 min	Step 7
9	Deploy Render web service + background worker.	5 min	Step 8
10	In Vercel: import <code>_oe</code> as a project. Set <code>VITE_*</code> env vars.	5 min	Step 9
11	In Cloudflare: add CNAME records for app. + api. subdomains.	5 min	Steps 9, 10
12	In Cloudflare: set SSL/TLS mode to "Full (Strict)".	1 min	Step 11
13	Visit <code>https://app.yourdomain.com</code> → login → take attendance.	5 min	Step 12

10.2 Render configuration

The `render.yaml` blueprint in the migration pack defines two services: a web service (the NestJS API, Standard plan at \$25/month for 2 GB RAM) and a background worker (the Graphile Worker process, also Standard plan). Both use the same Docker image — the only difference is the start command. The web service runs `node dist/main.js`; the worker runs `node dist/workers.js`.

Every environment variable marked `sync: false` in the blueprint must be set manually in the Render dashboard before the first deploy. These are the secrets — database credentials, JWT keys, R2 keys, Supabase keys, Stripe / SendGrid / Twilio API keys. Render encrypts them at rest and never logs them. Never commit these to git.

10.3 Vercel configuration

The `vercel.json` in the migration pack configures Vite as the framework, sets the build command to install and build `_oe`, and adds the security headers (X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy). The catch-all redirect to `index.html` ensures client-side routing works for any path.

Set two environment variables in the Vercel project settings: `VITE_API_URL` (your Render API URL, e.g. `https://api.yourdomain.com`) and `VITE_SUPABASE_URL` + `VITE_SUPABASE_ANON_KEY` (for the Realtime subscription). These are baked into the static build at compile time, so you must redeploy after changing them.

10.4 Cloudflare configuration

Cloudflare handles DNS, CDN, WAF, and DDoS protection. Add four CNAME records: `app` and `www` point to `cname.vercel-dns.com`; `api` points to your Render URL (`your-app.onrender.com`). Set the SSL/TLS mode to "Full (Strict)" so Cloudflare connects to your origin over HTTPS with certificate validation. Both Vercel and Render terminate TLS themselves, so this is the safest mode and prevents man-in-the-middle attacks between Cloudflare and your origin.

DNS PROXY MODE

Set proxy status to **DNS only** (grey cloud) for the `api` and `app` CNAMEs. Render and Vercel both have their own DDoS protection and CDN; proxying through Cloudflare adds latency without benefit. Use Cloudflare proxy (orange cloud) only for the root domain if you want Cloudflare to serve a landing page.

11. Observability — The First 48 Hours

A deployed system without observability is a ticking bomb. The first forty-eight hours after deployment are when most production incidents happen, and without monitoring you will not know about them until a user reports them. Set up the following before declaring the deployment done.

11.1 Uptime monitoring

Use the free tier of UptimeRobot or Better Stack. Point a monitor at `https://api.yourdomain.com/api/v1/health` (the liveness probe) with a one-minute interval, and another at `https://api.yourdomain.com/api/v1/health/ready` (the readiness probe that checks Postgres + Redis). Configure alerts to send an email and SMS to your phone. Set a 90-second grace period to avoid false positives during deploys.

11.2 Error tracking (Sentry)

Create a free Sentry account (5,000 errors per month on the free tier). Add the Sentry NestJS SDK to `_ae` and the Sentry React SDK to `_oe`. Set the `SENTRY_DSN` environment variable in both Render and Vercel. Configure Sentry to scrub PII (email addresses, phone numbers, payment tokens) before storing events — this is a NDPR (Nigeria Data Protection Regulation) requirement for handling student data.

11.3 Database backup verification

Supabase Pro includes daily backups with seven-day PITR. But a backup you have never restored is a hope, not a backup. Within the first 48 hours, do a test restore: in the Supabase dashboard, click "Restore to timestamp" and pick a point one hour ago. Restore into a new staging project (not your production project). Verify the row counts match. If they do not, you have a backup problem and you need to fix it before trusting the system with real student data.

11.4 Reboot test

Trigger a manual redeploy of the Render web service. The new instance should boot in under thirty seconds and start passing health checks. If it takes longer, your cold-start path is too slow — investigate before the first unplanned restart. Then trigger a manual reboot of the Render background worker and verify it picks up pending jobs from the Graphile queue without duplicates.

12. Cost Summary

The total monthly cost of the managed stack is approximately seventy-five to one hundred US dollars, depending on usage. The largest line items are Render (fifty dollars for API plus worker) and Supabase Pro (twenty-five dollars for Postgres with PITR). Cloudflare, Vercel, Upstash, and R2 are free or near-free at your scale.

Service	Plan	Base cost	Scaling trigger
Cloudflare	Free	\$0	Pro (\$20/mo) when you need Page Rules or image optimisation.
Cloudflare R2	Free tier	\$0–2	\$0.015/GB after 10 GB stored. Egress always free.
Vercel	Hobby (free)	\$0	Pro (\$20/mo) at 100 GB bandwidth or team features.
Render API	Standard	\$25	Add a second instance at ~500 schools (\$50/mo total).
Render Worker	Standard	\$25	Add a second worker at ~200 schools (\$50/mo total).
Supabase	Pro	\$25	Add-on for >8 GB DB or >100 GB storage.
Upstash	Free	\$0	Pay-as-you-go above 10k commands/day.
Domain	Annual	~\$10/yr	—
	Total	~\$75–100/mo	Breaks even at ~3 paying students (¥150/student/mo).

13. Operational Runbook — First Incidents

Pre-write the runbooks for the five incidents you will definitely have. These should be executed from a printed copy, not invented under stress. The cost of writing them now is one hour; the cost of inventing them during an incident is a multi-hour outage and lost customer trust.

13.1 Cross-tenant data leak suspected

- **Step 1:** Rotate all JWT keys immediately. Generate a new RSA keypair, set it in Render env vars, redeploy.
- **Step 2:** Force re-authentication for all users (bump `JWT_ISSUER` to invalidate existing tokens).
- **Step 3:** Query `audit_logs` for the suspected actor's actions in the last 7 days.
- **Step 4:** Identify the affected schools by checking the actor's queries against the `school_id` column.
- **Step 5:** Notify affected schools within 72 hours (NDPR requirement) with: what happened, what data was exposed, what you have done to fix it.
- **Step 6:** Post-mortem: identify the root cause (was RLS bypassed? was there a `SUPER_ADMIN` misuse? was there a missing policy?).

13.2 Database unavailable

- **Step 1:** Check Supabase status page (status.supabase.com) for an ongoing incident.
- **Step 2:** If Supabase is down: switch the Render env var `DB_HOST` to the standby URL (Supabase Pro provides HA failover).
- **Step 3:** If failover does not work: pause the Render worker to prevent job backlog, post a status update on your status page.
- **Step 4:** Communicate to all schools via email (use SendGrid directly, not your app) — they value transparency more than uptime.
- **Step 5:** After recovery, verify the audit log for any incomplete transactions.

13.3 Payment provider outage

- **Step 1:** Detect via webhook failure rate (Sentry alert on >5% of webhooks failing in 5 minutes).
- **Step 2:** Switch the active provider env var to the fallback (Stripe ↔ Paystack).
- **Step 3:** Mark all in-flight payments as "pending manual review" — do not auto-fail them.
- **Step 4:** Communicate to affected schools that payments may take 24 hours to settle.
- **Step 5:** After recovery, reconcile manually-marked payments against the provider's reports.

14. The 14-Day Self-Test

Before onboarding any real school, use the product yourself for fourteen days. Log in every morning at 8:05 AM (the thundering herd). Take attendance for a fake class of 30 students. Generate a report card. Upload a CSV. Trigger a payment. Send an announcement. Watch the Sentry feed for errors. If the error rate is not near zero after fourteen days, you are not ready for real customers.

PATTERN TO AVOID

The single biggest mistake first-time founders make is signing up a paying customer in week 1, then having a four-hour outage in week 2 because Elasticsearch ate all the RAM. The customer tells ten other schools. You are done. Take two weeks to actually use the product on the real domain before anyone else does.

Once you have run for fourteen days without an unscheduled restart, restored from a backup successfully, and seen the Sentry error rate drop to near-zero, you are ready to onboard your first school. Pick one school you trust, give them a free month, and shadow their usage closely. Their first week will surface three to five issues you did not catch. Fix them, then open the doors to the next ten.

For specific code questions about any file in the migration pack, refer to the comments at the top of that file — each file explains what it replaces, why, and how to verify it works.